

Article

Detecting Fake Reviews Using Aspect-Based Sentiment Analysis and Graph Convolutional Networks

Prathana Phukon ¹, Petros Potikas ² and Katerina Potika ^{1,*}

¹ Computer Science Department, San Jose State University, San Jose, CA 95192, USA; prathana.phukon@sjsu.edu

² Electrical and Computer Engineering School, National Technical University of Athens, 15780 Athens, Greece; ppotik@cs.ntua.gr

* Correspondence: katerina.potika@sjsu.edu; Tel.: +1-408-924-5134

Abstract: Online reviews significantly influence consumer behavior and business reputations. Detecting fake reviews is important for maintaining trust and integrity in these platforms. We present an aspect-based sentiment analysis approach, referred to as FakeDetectionGCN, to distinguish genuine feedback from deceptive content. The idea is to analyze sentiments related to specific aspects (features) within reviews. Graph convolutional networks are used to model the complex contextual dependencies in the review texts. Additionally, SenticNet, an external semantic resource, is integrated to enhance the understanding of sentiments in the reviews. This model is capable of identifying both human-generated (genuine) as well as computer-generated (fake) reviews. It has been evaluated on two types of datasets and has shown strong performance across both. Through this work, we contribute to the effective detection of fake reviews and maintaining a trustworthy online review ecosystem.

Keywords: fake reviews; aspect-based sentiment analysis; graph neural networks



Academic Editor: Douglas O'Shaughnessy

Received: 22 February 2025

Revised: 14 March 2025

Accepted: 24 March 2025

Published: 29 March 2025

Citation: Phukon, P.; Potikas, P.; Potika, K. Detecting Fake Reviews Using Aspect-Based Sentiment Analysis and Graph Convolutional Networks. *Appl. Sci.* **2025**, *15*, 3771. <https://doi.org/10.3390/app15073771>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Online reviews critically shape consumer purchasing behavior and brand reputation, serving as trusted decision-making tools. However, the surge in fake reviews on online platforms and social networks erodes this trust, creating challenges for buyers and businesses. These fake reviews distort consumer expectations about products and services, driving the development of detection methods. Current techniques predominantly rely on metadata, such as user online activity patterns or purchase history, which faces practical limitations due to restricted accessibility. Most graph-based methods for fake review detection model the reviews as nodes and solve a node classification problem [1] or an unsupervised problem [2].

This creates a pressing need for text content analysis, and identifying fake reviews through textual patterns alone remains an interesting challenge. Aspect-based sentiment analysis (ABSA) [3] addresses this gap by extending traditional sentiment analysis. Rather than assessing overall positive/negative sentiments, ABSA evaluates opinions about specific product attributes (e.g., “Excellent battery life but unusable camera”). This granular approach exposes inconsistencies—like contradictory assessments of related features or exaggerated praise for minor aspects that often signal fake generated reviews. By analyzing sentiment at the attribute level rather than depending on external metadata, ABSA provides a more viable approach for detecting fake online reviews.

This paper presents a new method called FakeDetectionGCN for detecting fake reviews. Each review is represented as a graph, and the main idea is to perform graph classification (real or fake) using graph convolutional networks (GCNs) [4]. This approach is effective because it captures and analyzes the complex relationships between words within a review. We model each review as a graph, where nodes represent words and edges represent semantic relationships. A review may contain several aspect terms and sentiments. We aim to leverage ABSA to capture these aspect–sentiment relationships in our graph structure and improve the fake review detection process by discovering inconsistencies found between aspects and their sentiments in a review. Here is an example of a review: “The food is delicious, but the service is slow”. The aspects are food and service. The sentiments are delicious (positive) and slow (negative).

Figure 1 illustrates a simple example of how a review can be modeled as a graph. The aspect nodes are represented in blue and the sentiment words are in green. The edges connecting these nodes are weighted according to the sentiment.

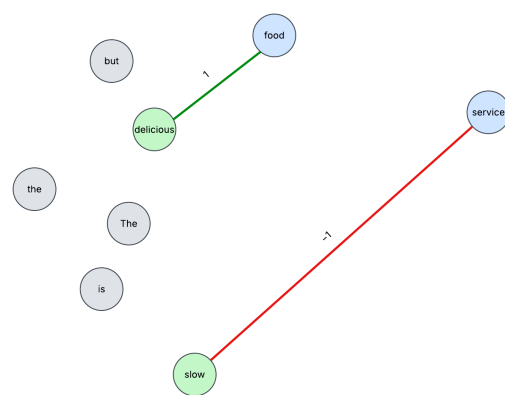


Figure 1. Example of a simple review graph.

The basic steps of FakeDetectionGCN are shown in Figure 2. We begin with data gathering and preprocessing to prepare the reviews for analysis. Following this, we perform feature extraction using various word embeddings, such as GloVe [5], BERT [6], and Word2Vec [7] for each review (graph). These embeddings capture the semantic details of the text. These are then used as the feature matrix for our GCN. We then create an adjacency matrix, where edge weights are adjusted based on the sentiment scores of the aspects.

The next phase involves training the graph convolutional network (GCN) model. The GCN model learns from the node features and the aggregated neighborhood information. The model is then evaluated based on its ability to accurately classify reviews as genuine or fake. Additionally, hyper-parameters are fine-tuned to optimize performance.

This paper is structured as follows: Section 2 reviews existing research on aspect-based sentiment analysis (ABSA) and the detection of fake reviews. Section 3 explains the background concepts that are used for the experiments. Section 4 describes the methodology used in the study. Section 5 consists of detailed results of the experiments and their outcomes, and Section 6 includes a comparative analysis. The report concludes with Section 7, which summarizes the findings and proposes ideas for future enhancements. Our code can be found here: <https://github.com/Pori-101/Fake-Review-Detection-using-Aspect-Based-Sentiment-Analysis-and-Graph-Convolutional-Networks> (accessed on 26 March 2025).

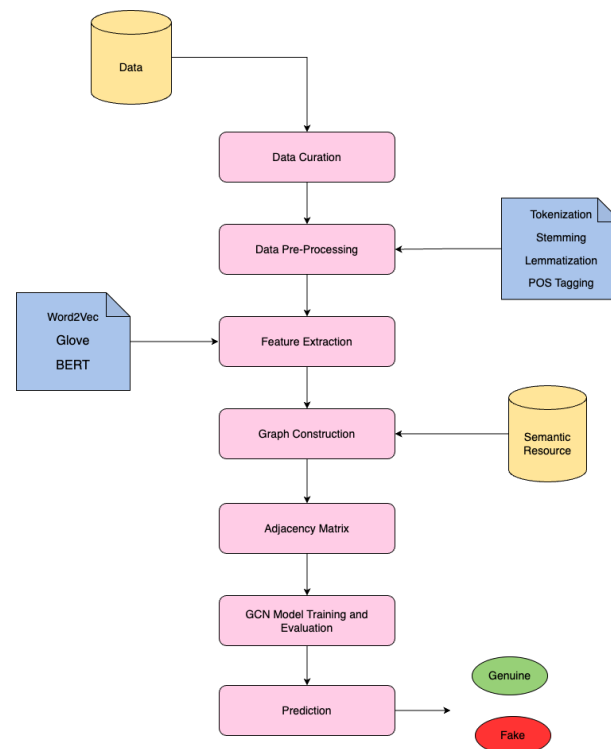


Figure 2. FakeDetectionGCN approach.

2. Related Study

This section covers the review of the literature in the field of aspect-based sentiment analysis and fake review detection.

2.1. Aspect-Based Sentiment Analysis

Aspect-based sentiment analysis (ABSA) is a subfield of sentiment analysis. It has become increasingly popular in recent years due to the need for more fine-grained analysis of sentiment in textual data [3]. The goal is to extract and analyze sentiment information at the aspect or feature level. This approach is very useful for businesses as it helps them understand customer opinions about various aspects of their products or services. By identifying areas that need improvement, businesses can make informed decisions to improve their products and services [8]. ABSA is an important area of research within natural language processing (NLP) because of the increasing demand for sentiment analysis in several domains such as e-commerce, social media, and customer reviews [9]. ABSA identifies aspects or features and the corresponding sentiment polarities of these aspects. For example, we have a sentence s consisting of n words such that $s = w_1, w_2, \dots, w_{a1}, \dots, w_{ak}, \dots, w_n$. Here, w_i and w_{ai} represent the i th contextual and aspect words, respectively. Each sentence can have one or more aspects associated with different sentiment polarities. These sentiments can be classified into three categories: positive, neutral, or negative [10]. The insights gained from ABSA are very valuable. They help in evaluating customer satisfaction, understanding consumer attitudes, and improving the overall consumer experience [11].

2.2. Approaches to Aspect-Based Sentiment Analysis

Several approaches have been developed for ABSA. Syntax-based methods use the structural information present in reviews to improve ABSA: the authors in [12] proposed a lexicon that can effectively capture domain-specific cues related to aspects of movie reviews. This method identifies aspect–sentiment pairs by using dependency patterns that connect aspects and sentiment words. However, this approach only covers the most

frequent aspect-sentiment relations. As a result, less frequent or uncommon relations are sometimes missed.

Other techniques include supervised and unsupervised learning approaches. Supervised methods rely on labeled training data to train models that can accurately predict aspect labels and sentiment. In [13], they propose a supervised approach using conditional random fields (CRFs) to extract opinions. Their model effectively identified aspects in both single- and cross-domain settings. An unsupervised aspect extraction approach [14,15] used long short-term memory (LSTM) networks to capture aspect-related patterns.

Similarly, ref. [11] proposed a model based on a deep memory network (DMN). It used a memory network to store and retrieve information about aspects and words. Then, this was used to generate aspect-level sentiment predictions.

2.3. Graph Convolutional Networks and Their Application in ABSA

Graph convolutional networks (GCNs) were introduced by Kipf [4]. GCNs are a type of neural network specifically designed to operate on graph data. They take as input an adjacency matrix and a feature node matrix. They are built on the concept of convolutional networks. GCNs apply convolutional operations on graph data, which allows for graph representation learning. GCNs learn node representations that capture both local and global graph structures. This is achieved by iteratively propagating and updating node features using information from neighboring nodes. GCNs have proven to be a very powerful tool for ABSA. They enable the modeling of relationships between different aspects and their associated sentiments.

Phan et al. [16] proposed Convolutional Attention Neural Network over Graph Structures (CANGS) for ABSA. Their model combines graph convolutional networks with attention mechanisms. It allows the model to focus only on relevant aspects and words for sentiment classification. Their experimental results show that CANGS outperforms several existing methods.

GCNs require a predefined graph structure, which can be challenging to construct. Sun et al. [17] introduced a method to construct a graph from a dependency tree for each sentence using a dependency parser. In this graph, the nodes represent the words of the sentence, and the edges reflect the syntactic dependencies between these words. Each node includes a feature vector with word embeddings, part-of-speech tags, and dependency labels. Aspect nodes are also added to the graph to represent aspects distinctly. These aspect nodes are connected to the sentiment words in the sentence that express the corresponding aspects. The authors use a predefined aspect dictionary to identify the words associated with each aspect.

SenticGCN [10] incorporates this dependency graph construction and proposes a GCN model to predict the sentiment of an aspect. This model integrates affective knowledge and emotions to improve the sentiment capture of each aspect. This is achieved by integrating emotional information into the GCN through an external resource called SenticNet [18]. This approach effectively captures emotional correlations between aspects and words. The model outperformed 25 existing approaches, making a significant contribution to the field.

Another recent work [19] proposed a novel weight-oriented GCN model. This model assigns different weights to aspects based on their importance in the graph. It uses both aspect and context embeddings to make aspect-level sentiment predictions.

2.4. Fake Reviews and Their Propagation

As defined in [20], fake reviews are written by individuals who are not genuine customers of the product or service. In some cases, the individuals may even be incentivized through rewards to leave feedback. The underlying goal is to skew the public perception

of a product or service. Such deceptive practices challenge the integrity of online review systems that have become very important across industries like e-commerce and hospitality. Initially, the intention was to provide transparency and guide potential consumers by offering genuine user feedback. But these platforms are now being increasingly exploited to manipulate public opinion. The goal of this is either to unfairly elevate or degrade the reputation of the offering. According to [21], the economic repercussions of such review manipulations are substantial. Their research indicates that even minor differences in a product's review scores can have significant impacts on consumer behavior. This, in turn, impacts the success or failure of a business. Even though fake reviews try to copy real reviews, they still can be distinguished. A study conducted in 2011 [22] showed that fake reviews tend to exhibit distinct linguistic patterns. These patterns can help differentiate them from genuine reviews upon closer examination.

Fake reviews can be of two types: paid human-written content or bot-generated reviews. Researchers have [23] identified unique patterns in review metadata that could help detect fake reviews. Metadata such as timing and frequency of posts could indicate fraudulent activity. Having stronger verification systems on online platforms can help reduce the number of fake reviews.

Various methods have been created to identify and minimize the impact of fake reviews. These methods range from simple rule-based algorithms to advanced machine learning (ML) techniques. Earlier methods were based on automated rules that looked for specific keywords and patterns in reviews. However, these initial methods often struggled with accuracy and scalability issues. As pointed out in [23], the deceptive practices used in review systems are complex. They require more sophisticated detection strategies. These strategies must be able to adapt as the tactics employed by creators of fake reviews continue to evolve and change over time. Rigid, rule-based approaches quickly became outdated against the dynamic nature of opinion spam and manipulation.

In addition to traditional ML techniques, deep learning approaches have become popular for detecting fake reviews. They can capture subtle details in text through layers of neural networks. Recurrent neural networks (RNNs) and convolutional neural networks (CNNs) have proven particularly effective. These models excel in analyzing semantic and syntactic structures in review texts to identify inconsistencies typical of fake reviews. In [24], they demonstrated the use of deep learning models that outperform traditional machine learning models. They integrated contextual embeddings that capture more complex dependencies in text data.

Although ABSA has been widely used for sentiment analysis, its application in identifying fake reviews has only recently been explored. A recent study [25] introduced a new approach that applies ABSA to detect fake reviews.

They propose using aspect replication instead of full-text replication for detecting fake reviews. The method is based on the observation that spammers often lack detailed knowledge about the product or service. They tend to manipulate the sentiment of specific aspects rather than the entire content. With their approach, they can distinguish how real and fake reviews are constructed. They employ a hybrid neural network model, combining convolutional neural networks (CNNs) and long short-term memory (LSTM) networks.

A survey that presents a taxonomy and covers advancements in three categories, fake review detection, fake reviewer detection, and fake review analysis, is [26]. Different kinds of fake reviews and their corresponding examples are also summarized. Furthermore, it discusses graph learning methods, including supervised and unsupervised learning approaches for fake review detection.

Some other surveys are presented in [1,27], which present the task of detecting fake reviews and summarize the existing datasets and their collection methods. They analyze

existing feature extraction techniques and summarize and critically analyze existing techniques to identify gaps based on two groups: traditional statistical machine learning, deep learning methods, and graph-based methods.

Finally, ref. [2] tries to detect groups of fake reviewers using an unsupervised method. Their approach can be summarized into two steps. First, cohesive groups are detected with the use of modularity-based graph convolutional networks. Then, the suspiciousness of each group is measured by several anomaly indicators, both at the individual and group levels. Fake reviewer groups can finally be detected through suspiciousness. All the graph-based papers mentioned use a different graph model than ours: the reviews are nodes, whereas for us, the reviews are graphs.

A recent research paper [28] introduced MBO-DeBERTa, a novel deep neural network with a Monarch Butterfly Optimizer, designed to detect fake reviews with improved accuracy and efficiency. The model also demonstrates resilience against adversarial attacks. Their two datasets are the ones we use.

This work aims to further explore the utilization of aspect-based sentiment analysis (ABSA) for detecting fake reviews. By extracting aspects and their respective sentiments, we aim to enhance the accuracy and efficiency of fake review detection. Building on the successful deployment of SenticGCN [10] for detecting the sentiment of an aspect, this paper adopts its architecture for fake review detection. We integrate the methodologies of [25] into our framework, leveraging their insights to improve our approach to identifying and analyzing fake reviews.

3. Background and Terminology

A graph $G = (V, E)$ consists of nodes V and edges E that can model relationships and interactions within different types of data. Nodes may have associated feature sets, and edges can have weights that signify the strength of connections between nodes.

A review is defined as a sequence of n words, denoted by

$$s = \{w_1, w_2, \dots, w_{a_1}, w_{a_2}, \dots, w_{a_k}, \dots, w_n\}.$$

Here, w_i represents the i -th contextual word in the review, and w_{a_i} represents the i -th aspect word within the review. Reviews often include multiple aspects, each of which may exhibit a sentiment polarity: positive, negative, or neutral, with values ranging from $[-1, 1]$ [10].

Word embeddings provide a way to represent textual data as vectors, where words that have similar meanings have a similar representation. The words are encoded in a way that similar words carrying similar contextual meanings are closer to each other in the vector space. This significantly improves the performance of natural language processing (NLP) models. We use Word2Vec [7], GloVe [5], and BERT [6].

We use the following neural networks: Long short-term memory (LSTM) [29] networks and graph convolutional networks (GCNs) [4]. LSTM is a type of recurrent neural network (RNN) specially designed to address the problem of long-term dependencies in sequence data and we use it to update the initial feature vector obtained from applying the word embedding technique to a review $h^{(0)}$. Once we have the node feature matrix $H^{(0)}$, we can feed it to the GCN. The second input for the GCN is the adjacency matrix A , which captures the connectivity between nodes.

4. Methodology

This section discusses in detail the methodology adopted in the paper. Figure 3 shows the model architecture of our FakeDetectionGCN. We next describe all the components.

Given a set of k reviews, we create an undirected weighted graph $G = (V, E)$ for each review and label each review accordingly as true or fake. Each node $v \in V$ is an aspect word of the review, and edges are dependencies between words.

We formulate our problem as a graph classification problem. Given k graphs with labels true or fake, predict the label for an unknown graph (review).

We accomplish three key tasks:

- Identify aspect terms (nouns representing aspects of a product/service) in the reviews and determine the associated sentiments for each aspect.
- Construct a graph that represents the different components of each review and how they are semantically related to each other.
- Classify the entire review graph as either real or fake. This is a binary classification, where 1 means the review is real, and 0 means it is fake.

So, in essence, we take the review text, identify the aspect terms and sentiments, build a graph capturing the review's semantics, and then, using the graph representation that we obtain with max pooling of the hidden nodes' representations, predict whether the review itself is real or fake.

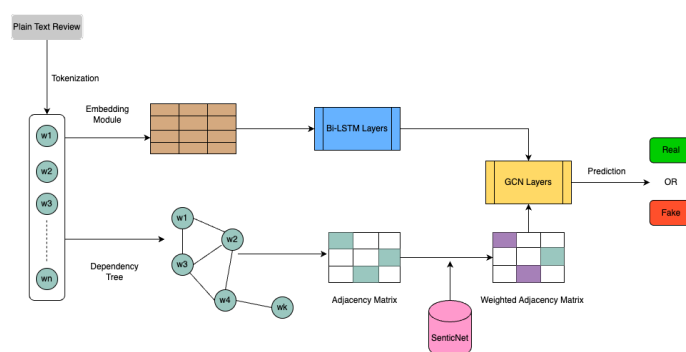


Figure 3. Model architecture of FakeDetectionGCN.

4.1. Tokenization

The first step in preprocessing involves tokenizing the reviews using the Spacy [30] NLP library. Each review r is broken down into its constituent words, represented as $r = \{w_1, w_2, \dots, w_n\}$, where each w_i is a token in the review.

4.2. Graph Construction

Next, we construct a graph using Spacy's dependency-parsing capabilities. These include part-of-speech (POS) tagging and lemmatization. One graph is created for each review, starting with the dependency tree. In this graph, the nodes represent tokens. Edges are added between nodes if the tokens have a semantic dependency, creating an undirected graph, $D_{ij} = D_{ji}$. This relationship is encoded in an adjacency matrix A , where $A_{ij} = D_{ij} = 1$ if there is a dependency between w_i and w_j , and 0 otherwise.

The adjacency matrix is then converted into a weighted adjacency matrix. This weighted matrix incorporates information about the aspect terms and their corresponding sentiments in the review text. If a term is a noun, we consider it a possible aspect and increase the weight of its connections. Additionally, we assign a sentiment score between -1 and 1 to each token based on SenticNet7 [18]. SenticNet7 is an artificial intelligence (AI) framework designed to improve sentiment analysis by using commonsense reasoning. It helps better interpret emotions in the text by applying a structured, knowledge-based approach. The weights are determined as follows:

$$T_{ij} = 1 \text{ if either word } (w_i) \text{ or } (w_j) \text{ is a noun, else } 0$$

$$S_{ij} = \text{SenticNet}(w_i) + \text{SenticNet}(w_j)$$

The final weights are computed, and the adjacency matrix is updated using the formula in [10]:

$$A_{ij} = D_{ij} \times (S_{ij} + T_{ij} + 1) \quad (1)$$

where D_{ij} indicates the presence of a semantic or syntactic dependency, S_{ij} is the sum of sentiment scores of the words, and T_{ij} increases the weight if either word is a noun.

Essentially, the affective S_{ij} scores detect unnatural sentiment shifts between a pair of words, a key trait of fake reviews. The T_{ij} score's intuition is that if the word is a noun it is more likely to be an aspect word, thus aspect-related words influence the graph learning process. Equation (1) increases the weight of the existing dependency edges; the plus one makes the term inside the parenthesis always bigger than 1 and does not diminish the dependency edges.

Algorithm 1 shows how the weighted adjacency matrix is computed. The time complexity of the algorithm is $O(n^2)$, given that we precomputed and saved the SenticNet scores and the information on whether a word is a noun or not. The total time for all graphs is $O(kn^2)$.

Algorithm 1 Construct Weighted Adjacency Matrix

```

1: Input: Plain Text Review
2: Output: Weighted adjacency matrix  $A$ 
3: Initialize graph  $G$  with empty nodes and edges
4: Parse the text input using Spacy to obtain tokens with their POS tags and dependency
   relations
5: for each token  $t_i$  in the parsed document do
6:   Add token  $t_i$  as a node in graph  $G$ 
7:   for each dependency from token  $t_i$  to token  $t_j$  do
8:     Add an undirected edge between node  $t_i$  and node  $t_j$  in graph  $G$ 
9: Initialize adjacency matrix  $A$  with dimensions equal to the number of tokens, set all
   entries to 0
10: for each edge  $(t_i, t_j)$  in graph  $G$  do
11:   Set  $A[i][j] = 1$  and  $A[j][i] = 1$ 
12: Transform  $A$  into a weighted adjacency matrix:
13: for each pair of tokens  $(t_i, t_j)$  connected in  $A$  do
14:   if either  $t_i$  or  $t_j$  is a noun then
15:      $T_{ij} = 1$ 
16:   else
17:      $T_{ij} = 0$ 
18:    $S_{ij} = \text{SenticNet}(t_i) + \text{SenticNet}(t_j)$ 
19:    $A[i][j] = A[j][i] = A[i][j] \times (S_{ij} + T_{ij} + 1)$ 
20: return the weighted adjacency matrix  $A$ 

```

4.3. Node Features

Initially, each token is transformed into a 300-dimensional vector using the pre-trained GloVe embeddings. This results in an embedding matrix where each row corresponds to a token in the review.

We derive the initial node feature matrix $H^{(0)}$ for the GCN using the Bi-LSTM model. This model takes in GloVe embedding vectors as input. It produces a refined feature matrix with 300 columns and n rows, where n is the number of tokens in the review text. The Bi-LSTM model captures contextual information from both forward and backward directions within the review text. This contextual information is encoded into the refined

feature matrix. We use a two-layer Bi-LSTM model with a mean squared error (MSE) loss function. The goal is to minimize the error between the input embeddings and the output embeddings produced by the model. In this way, we refine the initial GloVe embeddings to better represent the context of the review.

4.4. Data Preparation

The dataset is made up of graphs. Each graph has n nodes and m edges. The features of each node are represented as a matrix with 300 columns and n rows (one row per node). We divide this dataset into three parts: training set (80% of the data), validation set (10%), and test set (10%).

4.5. GCN Training

The last step is to train a graph convolutional network (GCN). This network has three convolutional layers. Each layer captures different levels of information about the features of the nodes and their neighbors. To prevent the model from overfitting, we add a dropout layer after the first two convolutional layers. After the convolutional layers, the network uses global average pooling. Global average pooling takes the average of the node features to create a singular representation for the entire graph. This step is important because our goal is to classify reviews (graphs) as either real or fake. We train the GCN model to optimize a loss function called negative log-likelihood loss. This loss function measures how different the predicted classes are from the true labels of the reviews. The GCNs are trained using backpropagation and gradient descent optimization algorithms. The weights of the network are gradually adjusted to minimize the loss function and improve the model's performance over time. The time and space complexity of GCNs is analyzed in [31].

4.6. Model Evaluation

To evaluate the model's performance, we look at two measures: validation loss and accuracy. We implement early stopping to prevent overfitting. Early stopping means we stop the training process when the validation loss does not improve for a certain number of epochs. This number is called 'patience'. For our experiments, we set the patience to 10 epochs. So if the validation loss does not improve for 10 epochs in a row, the training automatically stops. This helps ensure the model does not start overfitting to the training data.

To evaluate the performance of our model, we compute the following metrics [32]:

- **Accuracy:** This metric evaluates the overall correctness of the model. It is defined as the ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Observations}} \quad (2)$$

- **Precision:** This measures the accuracy of positive predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (3)$$

- **Recall (sensitivity):** This metric quantifies the ability of the model to find all the relevant cases (positive examples).

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (4)$$

- **F1 score:** The F1 score is the harmonic mean of precision and recall, providing a balance between them.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

- **Area under the curve (AUC):** The AUC represents the degree or measure of separability. It reflects the model's ability to distinguish between classes.

$$\text{AUC} = \int_0^1 \text{TPR}(t) dt \quad (6)$$

where $\text{TPR}(t)$ is the true positive rate at threshold t .

5. Experiments and Results

The method is implemented within the PyTorch [33] version 2.3.0 framework.

The GCN model architecture includes the following:

- A first convolutional layer (GCNConv) transforms node features from 300-dimensions to 64 dimensions.
- A second convolutional layer increases the feature dimensions from 64 to 128.
- A third convolutional layer further enhances the features to 256 dimensions.
- Global mean pooling is used to aggregate node embeddings to graph embeddings.
- A dropout layer with a rate of 0.5 follows the first and second convolutional layers. This reduces overfitting. We experimented with different dropout rates (0.2, 0.3, 0.5, 0.7).
- A fully connected layer maps the 256-dimensional features from the last convolutional layer to the number of classes. In this case, there are two classes, representing real or fake.

We used the Adam optimizer with a learning rate of 0.001 and a weight decay parameter of 5×10^{-4} .

The learning rate was chosen as 0.001 after the initial set of experiments.

5.1. Datasets

For our experiments, we use two datasets. The first one is the OTT dataset [34]. This dataset consists of 1600 reviews of 20 hotels located in Chicago. It has 800 truthful reviews collected from multiple platforms such as TripAdvisor, Expedia, Hotels.com, Orbitz, Priceline, and Yelp. It also consists of 800 fake reviews from Amazon Mechanical Turk. The data were created through crowd-sourcing, where the participants were asked to write fake reviews based on given instructions.

The second dataset was published in 2022 by Salminen et al. [35]. In their research paper, they developed a dataset to study the detection of fake reviews for online products. This dataset consists of a total of 40,000 reviews. It has 20,000 truthful reviews sourced from Amazon. The 20,000 fake reviews were computer-generated using GPT-2.

We experimented with different subsets of the datasets, see Table 1. Sample sizes were chosen based on the experiments of previous studies for comparative analysis.

Table 1. Description of the datasets.

Dataset	Total Reviews	True Reviews (%)	Fake Reviews (%)
OTT Dataset	1600	50%	50%
Salminen Dataset	40,000	50%	50%

The objective was to test the model's effectiveness in detecting both human-generated and computer-generated fake reviews.

5.2. Reproduction of Baseline Methods

In this study, we have reproduced two baseline models. The first is the SenticGCN model [10] for aspect-based sentiment analysis. Our proposed model (FakeDetectionGCN) for detecting fake reviews is based on their architecture. The second is the hybrid CNN-LSTM model [25] for fake review detection using ABSA.

5.2.1. SenticGCN for ABSA

The datasets utilized for this analysis are from the SemEval challenges. We use the Lap14, Rest14 [36], Rest15 [37], and Rest16 [38] datasets for the evaluation. They are considered benchmark datasets for aspect-based sentiment analysis. They contain reviews of laptops and restaurants.

The reproduction of the SenticGCN [10] model had similar results to the original study. The results are summarized in Table 2. It shows the comparison between the accuracy and F1 scores of the original vs. the reproduced values.

Table 2. Results of SenticGCN.

Dataset Name	Accuracy (%)		F1 Score (%)	
	Original [10]	Reproduced	Original [10]	Reproduced
Lap14	82.12	78.84	79.05	73.95
Rest14	86.92	85.89	81.03	79.32
Rest15	85.32	83.95	71.28	63.50
Rest16	91.97	90.58	79.56	73.02

For Lap14, Rest14, and Rest16, there is a slight decrease in both accuracy and F1 score for the reproduction. There is a significant difference in F1 score on the Rest15 dataset. This could be due to the difference in the initialization process and training environment.

5.2.2. CNN-LSTM for Fake Review Detection

For the second baseline, we tried to reproduce the fake review detection model of Bathla et al. [25]. Their model uses ABSA for fake review detection. We faced significant challenges in this reproduction as the source code was unavailable. It was also unclear how they carried out certain parts of their experiments, such as the extraction of aspect terms and classification of sentiments. This information was not mentioned in their paper. Hence, the attempt to reproduce their model was not successful.

The authors claim to use only aspect–sentiment pairs instead of entire reviews as input features for the CNN-LSTM model to enhance computational efficiency. We utilized the PyABSA [39] library to extract aspect terms and their corresponding sentiments for the experiments. However, the outcomes were not as expected.

Our observation was that the PyABSA library was not completely successful in extracting aspect terms and sentiments for all reviews. Moreover, for shorter reviews, only one aspect–sentiment pair, or sometimes even none, was available. These features as inputs were not sufficient to capture the entire review's structure for classification.

The model's performance achieved only about 60% accuracy, which is significantly lower than the 95.5% reported in the original study. Furthermore, the authors also performed their tests on the YelpChi [40] dataset, which is not available publicly for reproduction. Table 3 presents the reproduction results on the OTT dataset.

Table 3. Results of CNN-LSTM.

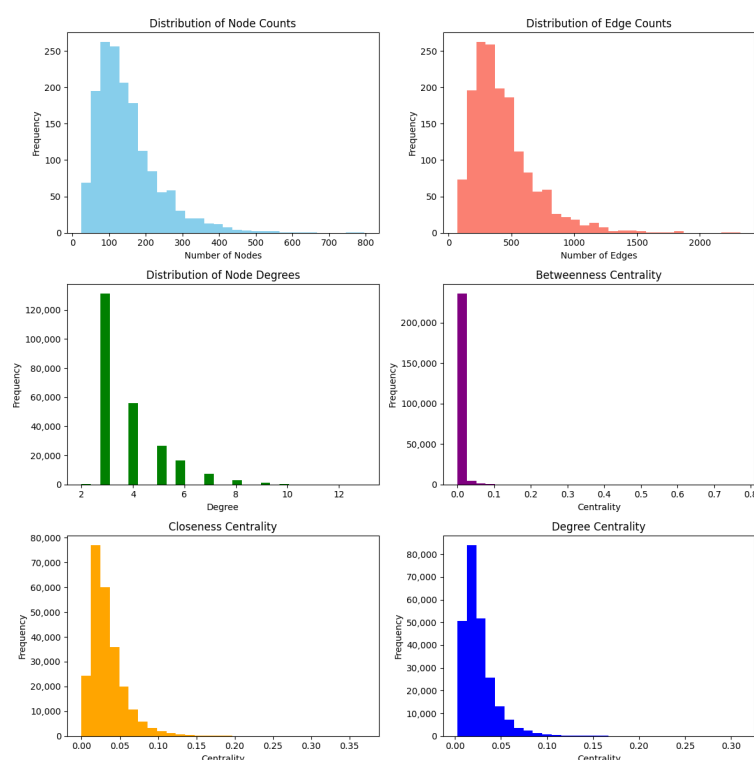
	Original [25]	Reproduced
Accuracy	95.5	60.14
Precision	89.5	58.33
F1 Score	87	57.8

5.3. Preliminary Analysis

For the preliminary analysis, we plot the graphs which are constructed using SenticNet and the dependency parsing of the review texts.

5.3.1. Graph Statistics of OTT Dataset

Figure 4 displays the distribution of graphs derived from the OTT dataset. The frequency distributions for both nodes and edges are right-skewed, indicating that most graphs have fewer nodes than edges. Notably, only a small number of graphs feature a significantly larger count. Additionally, centrality measures such as betweenness, closeness, and degree centrality generally have lower values. This suggests that many nodes exhibit limited connectivity and centrality. The analysis indicates that the graphs are characterized by a few highly interconnected nodes surrounded by many peripheral nodes with minimal centrality. These graphs show the properties of scale-free networks.

**Figure 4.** Statistics of graphs created from the OTT dataset.

5.3.2. Graph Statistics of the Salminen Dataset

Figure 5 illustrates the various distributions for graphs derived from the Salminen dataset with a sample size of 5000 (12.5%). The right-skewness in the histograms for both node and edge counts is more pronounced than in the OTT dataset. This suggests that the majority of graphs consist of fewer nodes than edges, while a minority displays substantially higher quantities. Similar to the OTT dataset, the centrality measures exhibit right-skewed distributions.

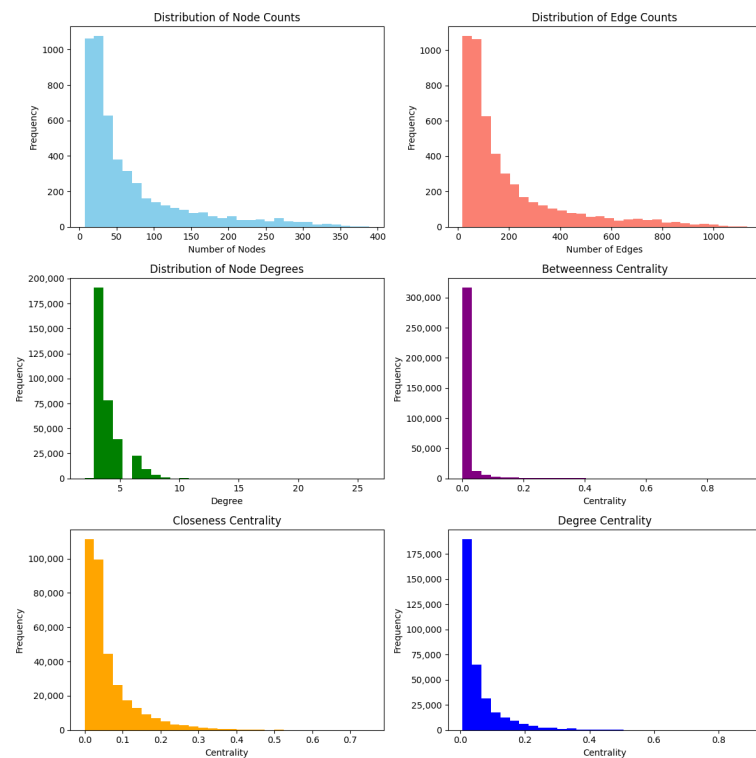


Figure 5. Statistics of graphs created from the Salminen dataset (12.5% sample).

Figure 6 presents various distributions for graphs derived from the Salminen dataset with a sample size of 10,000 (25%). This dataset exhibits trends similar to the other sample, reflecting consistency across different sample sizes.

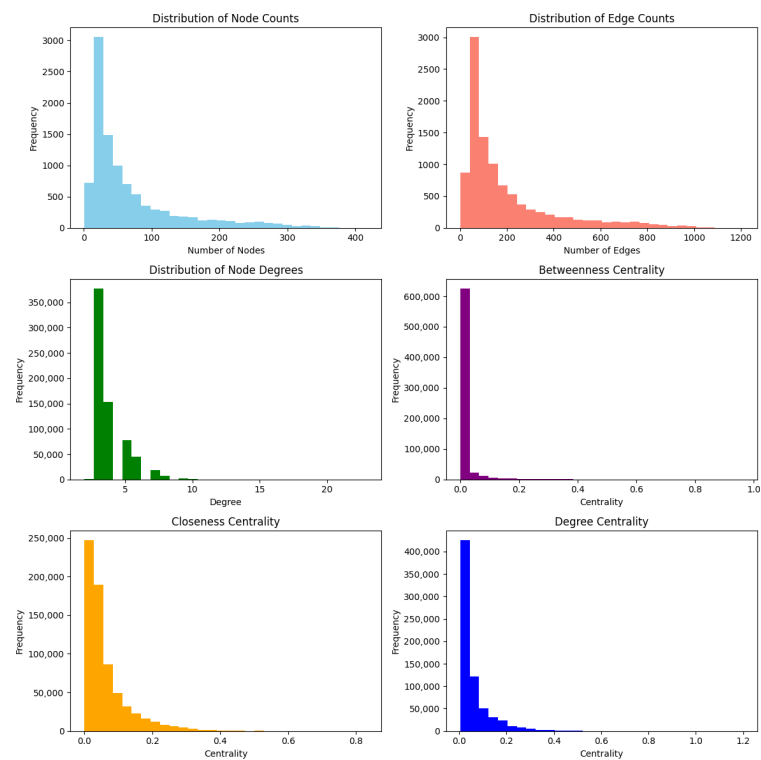


Figure 6. Graph statistics of Salminen dataset (25% sample).

5.4. Results of Traditional Classifiers

The two datasets were analyzed using traditional classifiers [41]. These classifiers are Random Forests, Support Vector Machine (SVM), Logistic Regression, Naïve Bayes, and a Multi-Layer Perceptron (MLP). These classifiers were tested with different word embeddings of the review texts. The word embeddings used were Word2Vec, GloVe, and BERT. The word embeddings were created by averaging them at the review level. The classifiers were trained and validated using 5-fold cross-validation. They were tested on a 20% holdout set.

5.4.1. OTT Dataset

Table 4 shows the results of the traditional ML classifiers on the OTT dataset. BERT embeddings consistently yielded superior results. The MLP classifier achieved the highest test accuracy of 0.8688 with BERT embeddings. The SVM using BERT embeddings also performed exceptionally. It achieved the highest accuracy of 0.8438. Conversely, the least effective results were with Word2Vec embeddings. With Word2Vec embeddings, the MLP had the lowest test accuracy of 0.6875.

Table 4. Results of traditional classifiers on the OTT dataset.

Model	Validation Accuracy	Test Accuracy
Word2Vec Embeddings		
Random Forest	0.7070	0.7031
Support Vector Machine	0.6734	0.7062
Logistic Regression	0.6891	0.7062
Naïve Bayes	0.6641	0.7062
Multi-Layer Perceptron	0.6539	0.6875
GloVe Embeddings		
Random Forest	0.7461	0.7781
Support Vector Machine	0.8008	0.8281
Logistic Regression	0.7797	0.8219
Naïve Bayes	0.7102	0.7094
Multi-Layer Perceptron	0.7906	0.8219
BERT Embeddings		
Random Forest	0.7781	0.7906
Support Vector Machine	0.8180	0.8438
Logistic Regression	0.8328	0.8438
Naïve Bayes	0.7383	0.7125
Multi-Layer Perceptron	0.8180	0.8688

5.4.2. Salminen Dataset

In the 12.5% sample of the Salminen dataset, GloVe embeddings combined with the MLP classifier delivered the best performance, reaching a test accuracy of 0.871. BERT embeddings also performed well. With BERT embeddings, the SVM and MLP classifiers achieved test accuracies of 0.865 and 0.866, respectively. The least effective was the Word2Vec embeddings with Random Forest. This combination only managed a test accuracy of 0.754. These results are listed in Table 5.

The results in Table 6 suggest that increasing the sample size from 5000 to 10,000 had little impact on model performance. It led to slightly higher accuracies. With the 25% sample of the Salminen dataset, GloVe embeddings continued to lead to improvements in performance. The MLP classifier achieved a test accuracy of 0.8675 with GloVe embeddings. The SVM with GloVe embeddings also showed strong results. It had a test

accuracy of 0.865. The least effective results came from the Naive Bayes classifier using Word2Vec embeddings. This classifier scored a test accuracy of only 0.671.

Table 5. Results of traditional classifiers on Salminen dataset (12.5%).

Model	Validation Accuracy	Test Accuracy
Word2Vec Embeddings		
Random Forest	0.741	0.754
Support Vector Machine	0.807	0.813
Logistic Regression	0.787	0.791
Naïve Bayes	0.739	0.757
Multi-Layer Perceptron	0.832	0.835
GloVe Embeddings		
Random Forest	0.839	0.832
Support Vector Machine	0.840	0.851
Logistic Regression	0.836	0.848
Naïve Bayes	0.791	0.792
Multi-Layer Perceptron	0.856	0.871
BERT Embeddings		
Random Forest	0.759	0.745
Support Vector Machine	0.850	0.865
Logistic Regression	0.844	0.859
Naïve Bayes	0.816	0.828
Multi-Layer Perceptron	0.845	0.866

Table 6. Results of traditional classifiers on Salminen dataset (25%).

Model	Validation Accuracy	Test Accuracy
Word2Vec Embeddings		
Random Forest	0.783	0.791
Support Vector Machine	0.80	0.81
Logistic Regression	0.771	0.780
Naïve Bayes	0.655	0.671
Multi-Layer Perceptron	0.833	0.836
GloVe Embeddings		
Random Forest	0.8354	0.8315
Support Vector Machine	0.8625	0.8650
Logistic Regression	0.8606	0.8610
Naïve Bayes	0.675	0.684
Multi-Layer Perceptron	0.8680	0.8675
BERT Embeddings		
Random Forest	0.7665	0.771
Support Vector Machine	0.858	0.854
Logistic Regression	0.8336	0.854
Naïve Bayes	0.6700	0.6685
Multi-Layer Perceptron	0.849	0.860

Overall, the results from the ML classifiers using different word embeddings show that GloVe embeddings combined with MLP classifiers performed the best. On the other hand, Word2Vec was the least effective at representing word vectors. MLP was the top-performing ML classifier, while Naïve Bayes performed the worst. BERT embeddings and SVM also demonstrated strong performances.

5.5. Results of FakeDetectionGCN Model

Figure 7 shows the training loss for the 2-layer Bi-LSTM model applied to the two datasets. As mentioned earlier, the FakeDetectionGCN method computes the initial node features from the 300-dimensional GloVe vectors of each token. These features are subsequently refined into 300-dimensional embeddings, which serve as input for a GCN. The objective of the model is to minimize the loss between the input and output embeddings, ensuring the embeddings are representative of the original inputs.

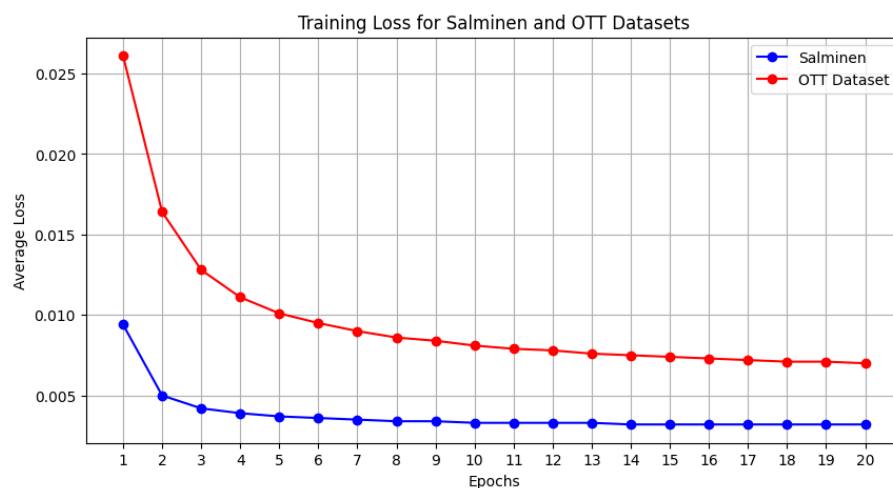


Figure 7. Loss curves for computation of node feature matrix.

From the curves, we observe a steady decrease in training loss, indicating consistent learning across both datasets. The Salminen dataset shows a smoother and more rapid convergence compared to the OTT dataset. This difference could be due to the variation in dataset complexity and size.

Table 7 summarizes the results of the FakeDetectionGCN model for different sample sizes of the two datasets. Figures 8–10 display the confusion matrices and PR curve plots of the datasets for visualizing the results.

Table 7. Evaluation results for OTT and Salminen datasets.

Dataset	Accuracy	Precision	Recall	F1 Score	AUC
OTT (80%)	0.8812	0.9054	0.8481	0.8758	0.94
OTT (100%)	0.900	0.8933	0.8933	0.8933	0.95
Salminen (12.5%)	0.8960	0.8885	0.9094	0.8988	0.95
Salminen (25%)	0.9070	0.9171	0.9086	0.9128	0.96
Salminen (100%)	0.9233	0.9333	0.9152	0.924	0.97

The OTT dataset at 80% sample size achieved an accuracy of 88.12%. It had a precision of 90.54% and a recall of 84.81%. This resulted in an F1 score of 87.58% and an AUC of 0.94. These results show that the model is quite precise, but it could improve in identifying true positives (real reviews).

For the OTT dataset at 100% sample size, the model achieved an accuracy of 90.00%. Precision and recall were both 89.33%, leading to an F1 score of 89.33% and an AUC of 0.95. These figures indicate a balanced performance. They suggest that the model is effective in both identifying true positives and avoiding false positives as the data size increases.

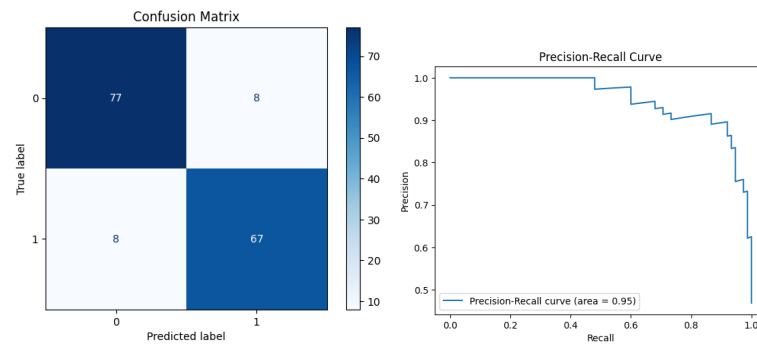


Figure 8. Results on the OTT dataset (100%).

For the Salminen dataset at 12.5% sample size, the accuracy was at 89.60%. The recall was 90.94%, which led to an F1 score to 89.88%. Although precision dropped slightly to 88.85% as compared to the OTT dataset, the overall performance improved, as shown by the AUC of 0.95.

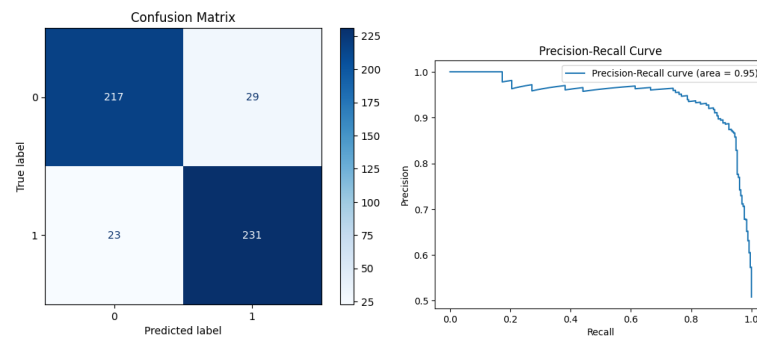


Figure 9. Results on Salminen dataset (12.5%).

With the Salminen dataset at 25% sample size, there were improvements across all metrics. Accuracy was 90.70%, and precision reached 91.71%. The recall was at 90.86%, leading to an F1 score of 91.28% and an AUC of 0.96.

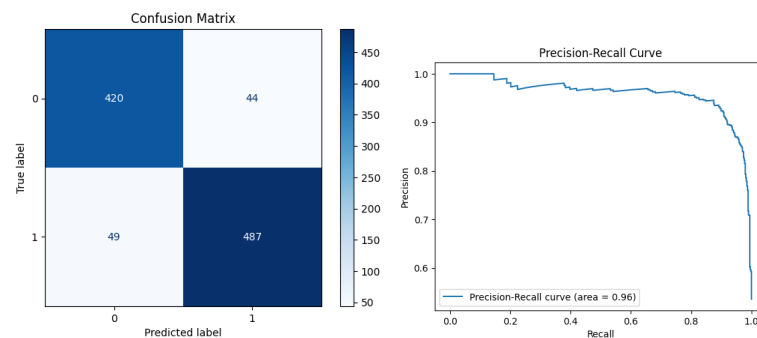


Figure 10. Results on Salminen dataset (25%).

In the Salminen dataset at 100% sample size, there was a further enhancement in performance. The accuracy reached 92.33%, and precision was very high at 93.33%. With a recall of 91.52%, the model had an F1 score of 92.40% and an AUC of 0.97.

Overall, a trend is observed across the datasets. Increasing the proportion of the data in training enhances the model's ability to generalize. It also helps the model perform more consistently across all metrics.

6. Discussion

For the analysis of our FakeDetectionGCN method's performance, we compare the results with those of our best-performing traditional classifier, as well as with the results from the baselines.

6.1. OTT Dataset

Figure 11 and Table 8 provide a comparative analysis of different models' performance on the OTT dataset. It highlights the performance metrics of FakeDetectionGCN against the traditional methods. These include the CNN-LSTM model [25] and the MLP classifier with BERT embeddings. The results focus on the 80% sample size. The reported results from [25] for this size are included.

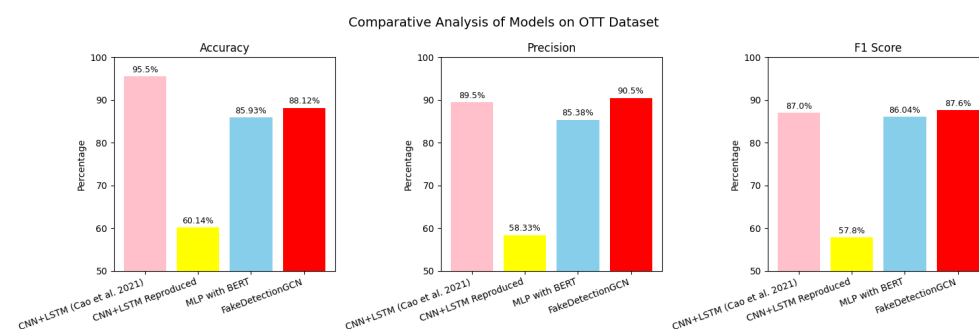


Figure 11. Bar Graph of model performance on OTT dataset (80%) [2].

The CNN-LSTM model reported an exceptionally high accuracy of 95.5%. However, our attempts to replicate these results were not successful. We achieved only about 60% accuracy. This raises questions about the reproducibility and reliability of the CNN-LSTM model under varying conditions or datasets.

FakeDetectionGCN has an accuracy of 88.12%. While this is not the highest, it is reliable. This is especially true considering the reproducibility issues with the Bathla model. FakeDetectionGCN also recorded the highest precision at 90.5% among the tested models. High precision indicates that the model effectively minimizes false positives.

Additionally, the F1 score of the FakeDetectionGCN model is 87.6%. This score is higher than that of the MLP with BERT. It is close to the CNN-LSTM's score, indicating a good balance between precision and recall.

Table 8. Comparative analysis on the OTT dataset (80%).

Model	Accuracy	Precision	F1 Score
CNN-LSTM [25]	95.5	89.5	87.0
MLP with BERT	85.93	85.38	86.04
MOB-DeBERTa [28]	87	87	87
FakeDetectionGCN	88.12	90.5	87.6

6.2. Salminen Dataset

Figure 12 and Table 9 provide a comparative analysis of performance metrics for the Salminen dataset. It compares FakeDetectionGCN to traditional models such as the crowd-sourced model, NBSVM, fakeRoBERTa, reported by [35] and the best-performing ML classifier.

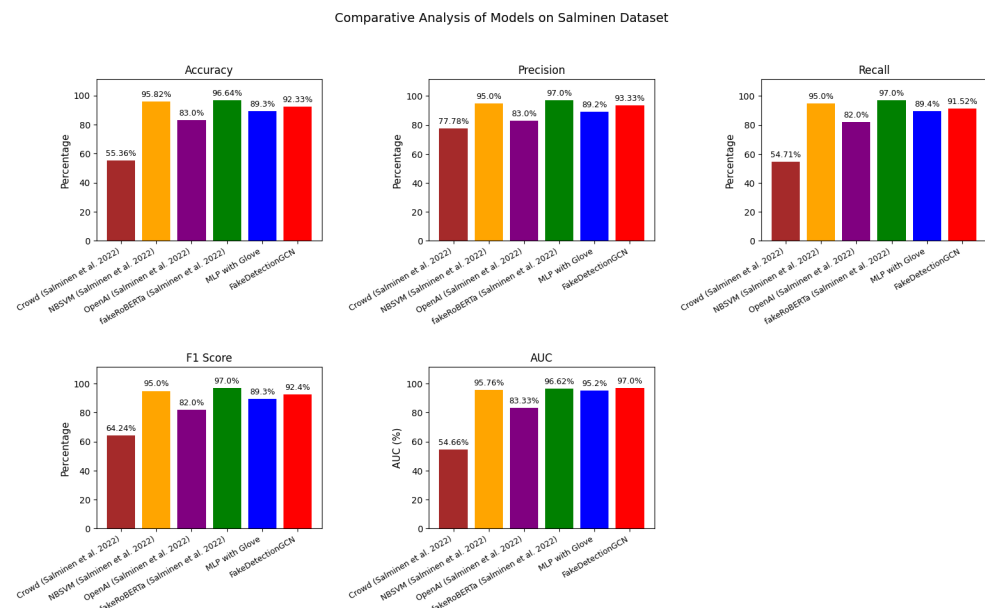


Figure 12. Bar graph of model performance on Salminen dataset (100%) [35].

The crowd-sourced model shows significantly lower performance. It has an accuracy of 55.36%, a precision of 77.78%, and an F1 score of 64.24%. These figures establish a basic benchmark for model performance on this dataset. In contrast, the NBSVM and fakeRoBERTa models exhibit exceptionally high performance. The NBSVM achieves an F1 score of 95%. fakeRoBERTa scores 97%.

Table 9. Comparative analysis on the Salminen dataset (100%).

Model	Accuracy	Precision	Recall	F1 Score	AUC
Crowd [35]	55.36	77.78	54.71	64.24	54.66
NBSVM [35]	95.82	95	95	95	95.76
OpenAI [35]	83	83	82	82	83.33
fakeRoBERTa [35]	96.64	97	97	97	96.62
MLP with GloVe	89.3	89.2	89.4	89.34	95.2
MOB-DeBERTa [28]	98	98	99	98	—
FakeDetectionGCN	92.33	93.33	91.52	92.4	97

FakeDetectionGCN demonstrates robust performance. It has an accuracy of 92.3%, a precision of 93%, and an F1 score of 91.5%. The AUC of 97% indicates superior capability in distinguishing between classes. This reflects the model's effectiveness in handling diverse and complex scenarios. In particular, the performance of FakeDetectionGCN is notably superior to that of the OpenAI model. The OpenAI model reported lower scores in all metrics, including an accuracy of 83% and an F1 score of 82%.

Salminen et al. [35] also tested their model's performance on the OTT dataset to evaluate its effectiveness in detecting human-written fake reviews. Figure 13 compares the AUC scores reported by [35] against FakeDetectionGCN.

It can be seen that the fakeRoBERTa and NBSVM models had near-perfect accuracy on the Salminen dataset, but their performance dropped drastically on the OTT dataset. This shows that their models are only effective in detecting computer-generated fake reviews and fail to detect human-written fake reviews accurately. In contrast, FakeDetectionGCN shows consistent and robust performance across all variations of the two datasets. The recent MBO-DeBERTa [28] shows much better results for the Salminen dataset.

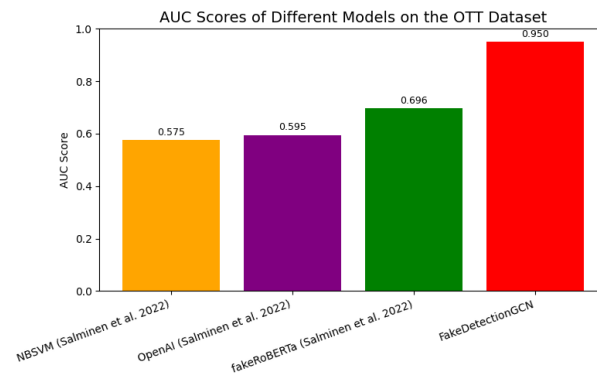


Figure 13. AUC scores of different models on OTT dataset (100%) [35].

7. Conclusions and Future Work

This paper highlights the promising potential of our FakeDetectionGCN model. This model uses aspect-based sentiment analysis and graph convolutional networks to detect fake reviews effectively. It handles both human-generated and computer-generated fake reviews well.

FakeDetectionGCN performs well when dealing with fake reviews created by advanced algorithms or manually written by individuals. This ability is key. It demonstrates the model can reliably distinguish genuine content from fake content across various data sources. By handling both types of fake reviews, our GCN model proves to be a strong tool against digital misinformation.

Despite its effectiveness, the model has some drawbacks. It requires long training times and involves significant computational complexity. These factors are challenging when resources or time are limited. The limitations are especially problematic in situations where quick processing is needed.

To improve the model's performance and efficiency, several areas can be explored:

- **Word embeddings:** Using different word embeddings could lead to better accuracy. Advanced embeddings like ELMo, BERT, or RoBERTa may capture context more effectively than traditional embeddings.
- **Hyper-parameter tuning:** Optimizing the hyper-parameters can reduce computational demands and enhance learning efficiency. Techniques like grid search or Bayesian optimization can find the best parameter settings.
- **Diverse datasets:** Testing the model on a wider variety of datasets, especially larger and more complex ones, is crucial. This will validate the model's applicability across contexts and improve its ability to handle real-world scenarios.
- **Reducing training time:** Decreasing the long training time and complexity is essential. More efficient training algorithms or hardware acceleration like GPUs could help. Simplifying the model architecture without affecting performance is another option.

In conclusion, while FakeDetectionGCN is promising for detecting fake reviews, there are opportunities to improve efficiency and scalability. Addressing these areas can make the model more practical for wider use.

Author Contributions: Conceptualization, P.P. (Prathana Phukon) and K.P.; methodology, P.P. (Prathana Phukon); software, P.P. (Prathana Phukon); validation, P.P. (Prathana Phukon), P.P. (Petros Potikas), and K.P.; formal analysis, K.P.; investigation, P.P. (Prathana Phukon); resources, P.P. (Prathana Phukon); data curation, P.P. (Prathana Phukon); writing—original draft preparation, K.P.; writing—review and editing, K.P.; visualization, P.P. (Prathana Phukon); supervision, K.P.; project administration, P.P. (Petros Potikas). All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data available in a publicly accessible repository.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ML	Machine learning
AI	Artificial intelligence
ABSA	Aspect-based sentiment analysis
POS	Part-of-speech
NLP	Natural language processing

References

1. Mohawesh, R.; Xu, S.; Tran, S.N.; Ollington, R.; Springer, M.; Jararweh, Y.; Maqsood, S. Fake reviews detection: A survey. *IEEE Access* **2021**, *9*, 65771–65802.
2. Cao, C.; Li, S.; Yu, S.; Chen, Z. Fake reviewer group detection in online review systems. In Proceedings of the 2021 International Conference on Data Mining Workshops (ICDMW), Auckland, New Zealand, 7–10 December 2021; pp. 935–942.
3. Liu, B.; Zhang, L. A survey of opinion mining and sentiment analysis. In *Mining Text Data*; Springer: Boston, MA, USA, 2012; pp. 415–463.
4. Kipf, T.N.; Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
5. Pennington, J.; Socher, R.; Manning, C. GloVe: Global Vectors for Word Representation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 25–29 October 2014; pp. 1532–1543.
6. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL), New Orleans, LA, USA, 1–6 June 2018.
7. Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G.S.; Dean, J. Distributed representations of words and phrases and their compositionality. In Proceedings of the Advances in Neural Information Processing Systems 26 (NIPS 2013), Lake Tahoe, NV, USA, 5–8 December 2013; Volume 26.
8. Birjali, M.; Kasri, M.; Beni-Hssane, A. A comprehensive survey on sentiment analysis: Approaches, challenges and trends. *Knowl.-Based Syst.* **2021**, *226*, 107134. [\[CrossRef\]](#)
9. Schouten, K.; Frasinca, F. Survey on aspect-level sentiment analysis. *IEEE Trans. Knowl. Data Eng.* **2015**, *28*, 813–830. [\[CrossRef\]](#)
10. Liang, B.; Su, H.; Gui, L.; Cambria, E.; Xu, R. Aspect-based sentiment analysis via affective knowledge enhanced graph convolutional networks. *Knowl.-Based Syst.* **2022**, *235*, 107643.
11. Tang, D.; Qin, B.; Liu, T. Aspect level sentiment classification with deep memory network. *arXiv* **2016**, arXiv:1605.08900.
12. Zhuang, L.; Jing, F.; Zhu, X.Y. Movie review mining and summarization. In Proceedings of the 15th ACM International Conference on Information and Knowledge Management, Arlington, VA, USA, 6–11 November 2006; pp. 43–50.
13. Jakob, N.; Gurevych, I. Extracting opinion targets in a single and cross-domain setting with conditional random fields. In Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, Cambridge, MA, USA, 9–11 October 2010; pp. 1035–1045.
14. Wang, Y.; Huang, M.; Zhu, X.; Zhao, L. Attention-based LSTM for aspect-level sentiment classification. In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, Austin, TX, USA, 1–4 November 2016; pp. 606–615.
15. Tan, M.; Santos, C.D.; Xiang, B.; Zhou, B. Lstm-based deep learning models for non-factoid answer selection. *arXiv* **2015**, arXiv:1511.04108.
16. Phan, H.T.; Nguyen, N.T.; Hwang, D. Convolutional attention neural network over graph structures for improving the performance of aspect-level sentiment analysis. *Inf. Sci.* **2022**, *589*, 416–439. [\[CrossRef\]](#)
17. Sun, K.; Zhang, R.; Mensah, S.; Mao, Y.; Liu, X. Aspect-level sentiment analysis via convolution over dependency tree. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 3–7 November 2019; pp. 5679–5688.

18. Cambria, E.; Liu, Q.; Decherchi, S.; Xing, F.; Kwok, K. SenticNet 7: A Neural Network Approach to Enhance Sentiment Lexicons. In Proceedings of the International Conference on Sentiment Analysis, Marseille, France, 20–25 June 2021.
19. Yu, B.; Zhang, S. A novel weight-oriented graph convolutional network for aspect-based sentiment analysis. *J. Supercomput.* **2023**, *79*, 947–972.
20. Mukherjee, A.; Venkataraman, V.; Liu, B.; Glance, N. What Yelp Fake Review Filter Might Be Doing? In Proceedings of the International AAAI Conference on Web and Social Media, Cambridge, MA, USA, 8–11 July 2013; Volume 7, pp. 409–418.
21. Luca, M.; Zervas, G. Fake It Till You Make It: Reputation, Competition, and Yelp Review Fraud. *Manag. Sci.* **2016**, *62*, 3412–3427. [\[CrossRef\]](#)
22. Ott, M.; Choi, Y.; Cardie, C.; Hancock, J.T. Finding Deceptive Opinion Spam by Any Stretch of the Imagination. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Portland, OR, USA, 19–24 June 2011; Volume 1, pp. 309–319.
23. Jindal, N.; Liu, B. Opinion Spam and Analysis. In Proceedings of the 2008 International Conference on Web Search and Data Mining, Palo Alto, CA, USA, 11–12 February 2008; pp. 219–230.
24. Li, J.; Ott, M.; Cardie, C.; Hovy, E. Towards AI-Complete Question Answering: A Set of Prerequisite Toy Tasks. *arXiv* **2020**, arXiv:1503.05623.
25. Bathla, G.; Singh, P.; Singh, R.K.; Cambria, E.; Tiwari, R. Intelligent fake reviews detection based on aspect extraction and analysis using deep learning. *Neural Comput. Appl.* **2022**, *34*, 20213–20229. [\[CrossRef\]](#)
26. Yu, S.; Ren, J.; Li, S.; Naseriparsa, M.; Xia, F. Graph learning for fake review detection. *Front. Artif. Intell.* **2022**, *5*, 922589.
27. Duma, R.A.; Niu, Z.; Nyamawe, A.S.; Manjotho, A.A. An analysis of graph neural networks for fake review detection: A systematic literature review. *Neurocomputing* **2025**, *623*, 129341.
28. Geetha, S.; Elakiya, E.; Kanmani, R.S.; Das, M.K. High performance fake review detection using pretrained DeBERTa optimized with Monarch Butterfly paradigm. *Sci. Rep.* **2025**, *15*, 7445.
29. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [\[PubMed\]](#)
30. Honnibal, M.; Montani, I. *spaCy: Industrial-Strength Natural Language Processing in Python*; Zenodo: Honolulu, HI, USA, 2020.
31. Blakely, D.; Lanchantin, J.; Qi, Y. Time and space complexity of graph convolutional networks. *Preprint* **2021**, *31*.
32. Scikit-Learn Developers. Model Evaluation. 2024. Available online: https://scikit-learn.org/stable/modules/model_evaluation.html (accessed on 29 April 2024).
33. PyTorch. Available online: <https://pytorch.org/> (accessed on 30 April 2024).
34. Ott, M.; Cardie, C.; Hancock, J. Estimating the prevalence of deception in online review communities. In Proceedings of the 21st International Conference on World Wide Web, Lyon, France, 16–20 April 2012.
35. Salminen, J.; Kandpal, C.; Kamel, A.M.; Jung, S.G.; Jansen, B.J. Creating and detecting fake reviews of online products. *J. Retail. Consum. Serv.* **2022**, *64*, 102771. [\[CrossRef\]](#)
36. Pontiki, M.; Galanis, D.; Pavlopoulos, J.; Papageorgiou, H.; Androutsopoulos, I.; Manandhar, S. SemEval-2014 Task 4: Aspect Based Sentiment Analysis. In Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014), Dublin, Ireland, 23–24 August 2014; pp. 27–35.
37. Pontiki, M.; Galanis, D.; Papageorgiou, H.; Manandhar, S.; Androutsopoulos, I. SemEval-2015 Task 12: Aspect Based Sentiment Analysis. In Proceedings of the 9th International Workshop on Semantic Evaluation (SemEval 2015), Denver, CO, USA, 4–5 June 2015; pp. 486–495.
38. Pontiki, M.; Galanis, D.; Papageorgiou, H.; Androutsopoulos, I.; Manandhar, S. SemEval-2016 Task 5: Aspect Based Sentiment Analysis. In Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval 2016), San Diego, CA, USA, 16–17 June 2016; pp. 19–30.
39. PyABSA Documentation. Available online: <https://pyabsa.readthedocs.io/> (accessed on 30 April 2024).
40. Rayana, S.; Akoglu, L. Collective opinion spam detection. In Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Sydney, Australia, 10–13 August 2015.
41. Scikit-Learn Developers. Supervised Learning. 2024. Available online: https://scikit-learn.org/stable/supervised_learning.html (accessed on 29 April 2024).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.